



utt  
UNIVERSITÉ DE TECHNOLOGIE  
TROYES

MEMBER OF



---

# Rapport de Projet

Prédiction d'insuffisance cardiaque

**IA01**

A23

TORREILLES Théo

BOCCARD Pierre-Emeric

Responsable de l'UE : Sandy MAHFOUZ ([sandy.mahfouz@utt.fr](mailto:sandy.mahfouz@utt.fr))

---

---

# Table des matières

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Table des matières.....</b>                            | <b>2</b>  |
| <b>I - Résumé du projet.....</b>                          | <b>4</b>  |
| a) Description du Projet.....                             | 4         |
| b) Aperçu du Dataset.....                                 | 4         |
| c) Contexte et Importance.....                            | 4         |
| d) Informations sur les Caractéristiques (Attributs)..... | 5         |
| e) Origine du Dataset et Références.....                  | 6         |
| f) Références.....                                        | 6         |
| <b>II - Analyse exploratoire des données.....</b>         | <b>7</b>  |
| a) Description des Colonnes.....                          | 7         |
| b) Statistiques descriptives.....                         | 8         |
| c) Présentation des cinq premières lignes.....            | 9         |
| d) Valeurs Manquantes et Nulles.....                      | 10        |
| e) Examen des Valeurs Aberrantes et Anormales.....        | 11        |
| 1) Valeurs constatées.....                                | 11        |
| 2) Vérification du Nombre de Valeurs Incohérentes.....    | 11        |
| f) Encodage des Caractéristiques Catégorielles.....       | 12        |
| <b>III - Approche considérée.....</b>                     | <b>13</b> |
| a) Étude des relations entre les caractéristiques.....    | 13        |
| b) Suppression des caractéristiques inutiles.....         | 14        |
| 1) Caractéristiques supprimées.....                       | 14        |
| 2) Caractéristiques gardées.....                          | 15        |
| c) Approche Train/Test.....                               | 15        |
| d) Normalisation des données.....                         | 16        |
| <b>IV - Différents choix d'implémentation.....</b>        | <b>17</b> |
| a) K-NN.....                                              | 18        |
| 1) Définition.....                                        | 18        |
| 2) Optimisation des hyper-paramètres.....                 | 19        |
| b) Random Forest.....                                     | 21        |
| 1) Définition.....                                        | 21        |
| 2) Optimisation des hyper-paramètres.....                 | 22        |
| c) Régression Logistique.....                             | 23        |
| 1) Définition.....                                        | 23        |
| 2) Optimisation des hyper-paramètres.....                 | 24        |

---

|                                           |           |
|-------------------------------------------|-----------|
| d) Réseau de neurones.....                | 26        |
| 1) Définition.....                        | 26        |
| 2) Optimisation des hyper-paramètres..... | 27        |
| e) SVC (Support Vector Machine).....      | 30        |
| 1) Définition.....                        | 30        |
| 2) Optimisation des hyper-paramètres..... | 31        |
| <b>V - Résultats obtenus.....</b>         | <b>32</b> |
| <b>VI - Analyse des résultats.....</b>    | <b>33</b> |

---

# I - Résumé du projet

## a) Description du Projet

Le projet vise à exploiter les techniques d'analyse de données et d'apprentissage automatique pour prédire les cas d'insuffisance cardiaque. Pour cela, nous utilisons un jeu de données rassemblant des informations provenant de différentes sources médicales. Ce jeu de données combine des ensembles de données sur les maladies cardiaques provenant de diverses origines, notamment l'hôpital de Cleveland, la Hongrie, la Suisse, Long Beach VA, et d'autres sources.

## b) Aperçu du Dataset

Le dataset se compose initialement de 1190 observations provenant de cinq ensembles de données différents. Après avoir identifié et éliminé les données en double, nous disposons finalement de 918 observations uniques pour notre analyse.

## c) Contexte et Importance

Les maladies cardiovasculaires demeurent la principale cause de décès à l'échelle mondiale, responsables d'un nombre estimé de 17,9 millions de vies perdues chaque année, soit environ 31% de tous les décès dans le monde. Parmi ces décès, quatre sur cinq sont dus à des crises cardiaques ou des AVC, et un tiers survient prématurément chez des individus de moins de 70 ans. L'insuffisance cardiaque, issue de ces maladies, est un événement fréquent et constitue l'objet de prédiction de notre étude.

Le dataset en question offre 12 caractéristiques pertinentes pour prédire de potentielles affections cardiaques. Cette prédiction précoce est cruciale pour les personnes atteintes de maladies cardiovasculaires ou à risque élevé, afin de mettre en place une gestion et un suivi adéquats.

---

## d) Informations sur les Caractéristiques (Attributs)

Le jeu de données comprend plusieurs caractéristiques essentielles telles que l'âge du patient, le sexe, le type de douleur thoracique, la pression artérielle au repos, le taux de cholestérol, la glycémie à jeun, les résultats de l'électrocardiogramme au repos, le taux maximal de battements cardiaques atteints lors de l'effort, la présence d'angine de poitrine induite par l'exercice, entre autres. Ces attributs sont fondamentaux pour l'élaboration de notre modèle de prédiction.

- Âge : L'âge du patient [années]
- Sexe : Le sexe du patient [M : Masculin, F : Féminin]
- Type de douleur thoracique : Type de douleur thoracique [TA : Angine typique, ATA : Angine atypique, NAP : Douleur non angineuse, ASY : Asymptomatique]
- Pression artérielle au repos : Pression artérielle au repos [mmHg]
- Cholestérol sérique : Taux de cholestérol sérique [mm/dl]
- Glycémie à jeun : Glycémie à jeun [1 : Si glycémie à jeun > 120 mg/dl, 0 : Sinon]
- Électrocardiogramme au repos : Résultats de l'électrocardiogramme au repos [Normal : Normal, ST : Présence d'anomalie de l'onde ST-T (inversions de l'onde T et/ou élévation ou dépression de ST > 0.05 mV), LVH : Montrant une hypertrophie ventriculaire gauche probable ou définitive selon les critères d'Estes]
- Fréquence cardiaque maximale atteinte : Fréquence cardiaque maximale atteinte [Valeur numérique entre 60 et 202]
- Angine de poitrine induite par l'exercice : Angine de poitrine induite par l'exercice [Y : Oui, N : Non]
- Dépression de ST (Oldpeak) : Dépression de ST = Ancien pic [Valeur numérique mesurée en dépression]
- Pente du segment ST au pic de l'exercice : La pente du segment ST au pic de l'exercice [Up : Ascendante, Flat : Plate, Down : Descendante]
- Maladie cardiaque : Classe de sortie [1 : Maladie cardiaque, 0 : Normal]

---

## **e) Origine du Dataset et Références**

Ce jeu de données a été créé en combinant cinq ensembles de données distincts portant sur les maladies cardiaques. Ces ensembles ont été soigneusement choisis pour fournir une variété de données et une ampleur substantielle pour la recherche. Les sources individuelles comprennent l'hôpital de Cleveland, des institutions médicales hongroises et suisses, le Long Beach VA, ainsi qu'un ensemble de données spécifique appelé Stalog (Heart) Data Set.

Ce rapport s'appuie principalement sur le jeu de données consolidé et épuré, mettant en évidence les efforts de diverses institutions médicales et chercheurs ayant contribué à la création et à la mise à disposition de ce précieux ensemble de données.

## **f) Références**

Le jeu de données original a été compilé par des chercheurs et institutions renommés, dont l'Institut hongrois de cardiologie, des hôpitaux universitaires en Suisse, le V.A. Medical Center de Long Beach et la Fondation Cleveland Clinic.

---

## II - Analyse exploratoire des données

Le jeu de données "Heart Failure Prediction" est composé de 918 entrées et 12 colonnes. Il se présente comme suit :

### a) Description des Colonnes

Afin de découvrir la composition du dataset et de comprendre comment les données sont entrées, nous avons fait la description du dataset. On y retrouve alors les colonnes citées dans le descriptif officiel du dataset, sous forme de valeurs numériques et catégorielles.

Le tableau ci-dessous présente les informations sur les colonnes du dataset :

| Colonne               | Non-Null Count | DType   |
|-----------------------|----------------|---------|
| <u>Age</u>            | 918 non-null   | int64   |
| <u>Sex</u>            | 918 non-null   | object  |
| <u>ChestPainType</u>  | 918 non-null   | object  |
| <u>RestingBP</u>      | 918 non-null   | int64   |
| <u>Cholesterol</u>    | 918 non-null   | int64   |
| <u>FastingBS</u>      | 918 non-null   | int64   |
| <u>RestingECG</u>     | 918 non-null   | object  |
| <u>MaxHR</u>          | 918 non-null   | int64   |
| <u>ExerciseAngina</u> | 918 non-null   | object  |
| <u>Oldpeak</u>        | 918 non-null   | float64 |
| <u>ST_Slope</u>       | 918 non-null   | object  |
| <u>HeartDisease</u>   | 918 non-null   | int64   |

---

## b) Statistiques descriptives

Afin d'analyser les données numériques à des fins de curiosité, nous avons procédé à des opérations de statistiques.

Les statistiques descriptives pour les caractéristiques numériques sont présentées ci-dessous :

| Caractéristique     | Moyenne | Écart-type | Minimum | 25ème percentile | Médiane | 75ème percentile | Maximum |
|---------------------|---------|------------|---------|------------------|---------|------------------|---------|
| <u>Age</u>          | 53.51   | 9.43       | 28.0    | 47.0             | 54.0    | 60.0             | 77.0    |
| <u>RestingBP</u>    | 132.40  | 18.51      | 0.0     | 120.0            | 130.0   | 140.0            | 200.0   |
| <u>Cholesterol</u>  | 198.80  | 109.38     | 0.0     | 173.25           | 223.0   | 267.0            | 603.0   |
| <u>FastingBS</u>    | 0.23    | 0.42       | 0.0     | 0.0              | 0.0     | 0.0              | 1.0     |
| <u>MaxHR</u>        | 136.81  | 25.46      | 60.0    | 120.0            | 138.0   | 156.0            | 202.0   |
| <u>Oldpeak</u>      | 0.89    | 1.07       | -2.6    | 0.0              | 0.6     | 1.5              | 6.2     |
| <u>HeartDisease</u> | 0.55    | 0.50       | 0.0     | 0.0              | 1.0     | 1.0              | 1.0     |



---

### c) Présentation des cinq premières lignes

Afin de visualiser la structure d'une ligne et son apparence, nous avons affiché les cinq premières lignes du dataset.

Voici un aperçu des cinq premières lignes du dataset :

| Age | Sex | ChestPainType | Resting BP | Cholesterol | Fasting BS | Resting ECG | MaxHR | Exercise Angina | Oldpeak | ST_Slope | Heart Disease |
|-----|-----|---------------|------------|-------------|------------|-------------|-------|-----------------|---------|----------|---------------|
| 40  | M   | ATA           | 140        | 289         | 0          | Normal      | 172   | N               | 0.0     | Up       | <u>0</u>      |
| 49  | F   | NAP           | 160        | 180         | 0          | Normal      | 156   | N               | 1.0     | Flat     | <u>1</u>      |
| 37  | M   | ATA           | 130        | 283         | 0          | ST          | 98    | N               | 0.0     | Up       | <u>0</u>      |
| 48  | F   | ASY           | 138        | 214         | 0          | Normal      | 108   | Y               | 1.5     | Flat     | <u>1</u>      |
| 54  | M   | NAP           | 150        | 195         | 0          | Normal      | 122   | N               | 0.0     | Up       | <u>0</u>      |

---

#### **d) Valeurs Manquantes et Nulles**

- Aucun doublon n'est présent dans le jeu de données (Elles ont été enlevées dans l'ensemble de départ par les créateurs du dataset).
- Aucune valeur manquante ou nulle n'est enregistrée pour aucune des caractéristiques.

---

## e) Examen des Valeurs Aberrantes et Anormales

### 1) Valeurs constatées

Les données ont été examinées pour identifier des valeurs potentiellement aberrantes dans différentes caractéristiques du dataset :

- Age : La plage de valeurs va de 28 à 77.
- Sexe : Il y a deux catégories 'F' et 'M'.
- ChestPainType : Il existe les valeurs 'ATA', 'NAP', 'ASY' et 'TA'.
- RestingBP : La plage de valeurs va de 80 et 200 mm Hg. On trouve des valeurs égales à 0.
- Cholestérol : Les valeurs de cholestérol varient de 85 à 603 mm/dl. On trouve des valeurs égales à 0.
- FastingBS : C'est une variable binaire (0 ou 1).
- RestingECG : Il y a trois catégories : 'Normal', 'ST', et 'LVH'.
- MaxHR : Les valeurs se situent entre 60 et 202.
- ExerciseAngina : C'est une variable binaire : 'N' ou 'Y'.
- Oldpeak : Les valeurs vont de -2.6 à 6.2.
- ST\_Slope : Il existe trois catégories : 'Up', 'Flat', et 'Down'.
- HeartDisease : C'est la classe de sortie, binaire (0 ou 1).

### 2) Vérification du Nombre de Valeurs Incohérentes

- RestingBP : Une seule valeur aberrante est observée avec une pression artérielle au repos de 0.
- Cholestérol : On relève 172 occurrences de valeurs aberrantes égales à 0 pour le taux de cholestérol.

Le reste des caractéristiques ne contiennent ni de valeurs aberrantes ni de valeurs incohérentes.

---

## f) Encodage des Caractéristiques Catégorielles

Afin de pouvoir exploiter les données catégorielles, nous avons encodé ces données.

Les caractéristiques catégorielles ont été encodées comme suit :

- LabelEncoder : Pour les catégories binaires
  - Sexe : Encodé en utilisant LabelEncoder avec F (0) et M (1).
  - ExerciseAngina : Encodé en utilisant LabelEncoder avec N (0) et Y (1).
- get\_dummies : Pour les catégories non-binaires
  - ChestPainType, RestingECG, ST Slope : Encodage en utilisant la méthode One-Hot Encoding.

Cela garantit que les données catégorielles sont correctement transformées en format numérique pour être utilisées plus tard dans nos algorithmes.

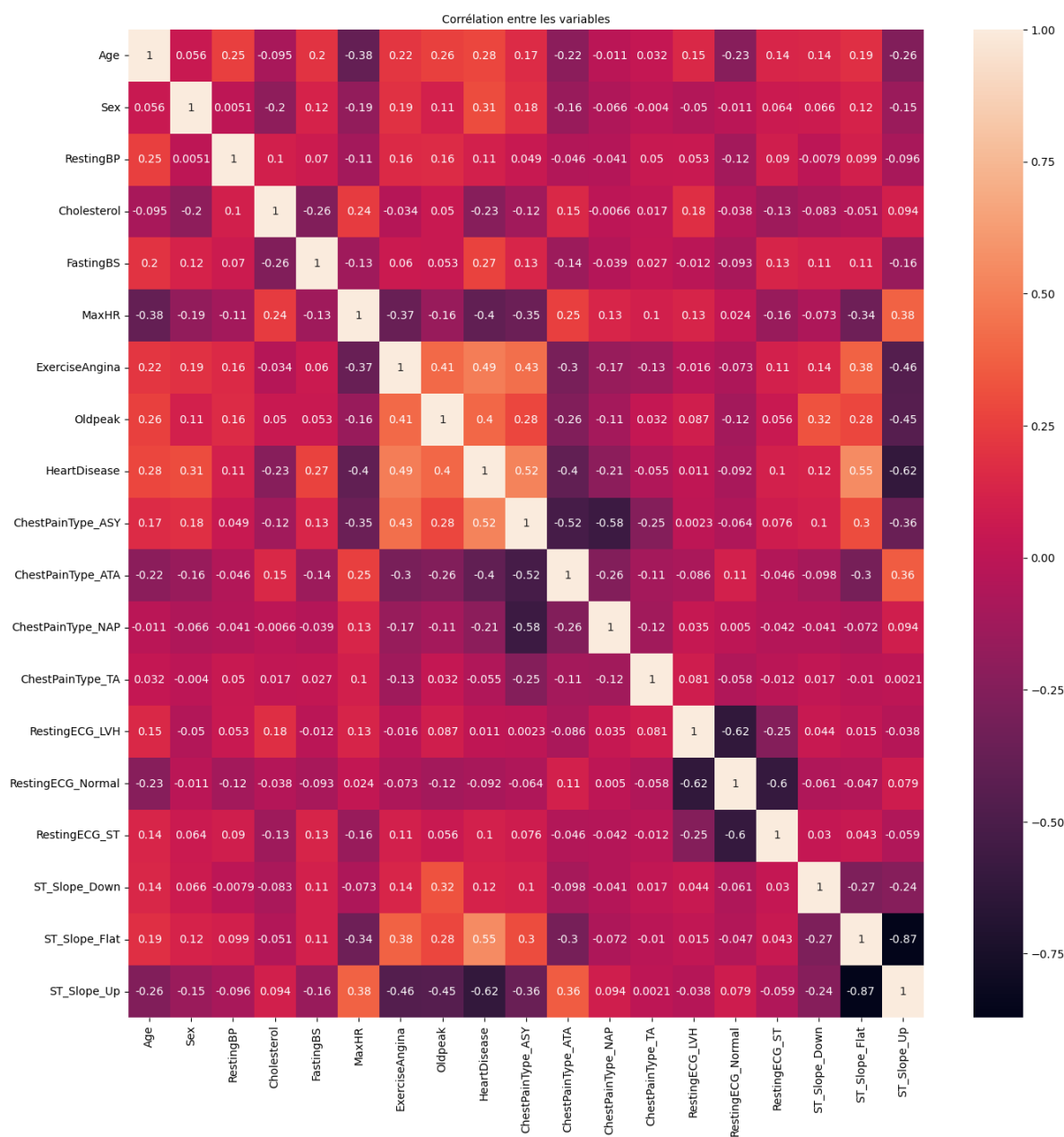
Toutes ces analyses démontrent que le jeu de données est complet et prêt pour une analyse plus approfondie.

### III - Approche considérée

#### a) Étude des relations entre les caractéristiques

Afin d'étudier ces relations, nous avons réalisé une matrice de corrélation, ce qui nous a permis de choisir les meilleures caractéristiques pour nos algorithmes.

Ci-dessous, la matrice de corrélation :



---

## **b) Suppression des caractéristiques inutiles**

À l'aide de la matrice de corrélation, nous avons décidé de supprimer des caractéristiques que nous avons jugées trop peu corrélées avec la caractéristique de sortie que nous voulons prédire, à savoir 'HeartDisease'.

Nous avons fixé un seuil à 0.2, ce qui signifie que toutes les caractéristiques avec une valeur de corrélation inférieure à 0.2 ont été supprimées ainsi que 'Cholesterol'. Il y a une exception à cette règle qui est le 'Cholestérol', qui a un coefficient de corrélation de -0.23. Or, en retirant les 172 valeurs incohérentes, le coefficient descendait à 0.1. Et comme RestingBP est en-dessous du seuil que nous avons fixé, on a donc fait le choix de ne pas supprimer de valeur, tout en supprimant la caractéristique du Cholestérol. Il ne nous est donc pas nécessaire de faire quelque opération de traitement supplémentaire pour pallier ces données incohérentes.

### **1) Caractéristiques supprimées**

Ci-dessous, les caractéristiques que nous avons supprimé :

- ChestPainType\_TA
- RestingBP
- Cholesterol
- RestingECG\_Normal
- RestingECG\_LVH
- RestingECG\_ST
- ST\_Slope\_Down

---

## 2) Caractéristiques gardées

Les caractéristiques gardées sont donc :

- Age
- Sex
- ChestPainType\_ASY
- ChestPainType\_ATA
- ChestPainType\_NAP
- FastingBS
- MaxHR
- ExerciseAngina
- Oldpeak
- ST\_Slope\_Flat
- ST\_Slope\_Up
- HeartDisease

### c) Approche Train/Test

Afin de pouvoir concevoir nos algorithmes et d'avoir des ensembles distincts pour nos entraînements et nos tests, nous avons divisé notre dataset en deux :

- Ensemble d'entraînement : 70% (X\_train, Y\_train)
- Ensemble de test : 30% (X\_test, Y\_test)

X contient toutes les caractéristiques gardées en partie III.b.2, à l'exception de 'Heart Disease' qui est déplacée dans Y.

---

## d) Normalisation des données

Nos données ne sont pas toutes à la même échelle. En effet, nous avons des catégories, des valeurs numériques, de grands écarts et les données n'étaient pas toutes situées sur la même échelle.

Pour palier à cela, nous avons effectué un scaling MinMax sur nos ensembles :

- $X$
- $X_{\text{train}}$
- $X_{\text{test}}$

On a choisi la fonction `MinMaxScaler()` car même si ce n'est pas nécessaire pour tous les algorithmes, c'est ce qui est privilégié pour les réseaux de neurones. Pour les autres il n'y a pas grande différence entre `MinMaxScaler` et `StandardScaler`. Ainsi, toutes nos données sont sur la même échelle et nous pouvons commencer les festivités.



---

## IV - Différents choix d'implémentation

Pour tous les algorithmes, nous avons choisi la métrique 'f1-score' comme métrique d'évaluation car elle combine la précision et le recall.

En effet, l'accuracy et la specificity ne sont pas des métriques intéressantes dans le cadre de ce projet car elles ne se basent pas sur des valeurs représentatives d'efficacité dans notre cadre. Du moins, pas assez pénalisantes.

- Accuracy : Nombre de prédictions correctes
- Precision : Proportion de classifications positives effectivement correctes
- Recall : Proportion de résultats positifs réels identifiés correctement
- Specificity : Proportion de résultats négatifs réels identifiés correctement

---

## **a) K-NN**

### **1) Définition**

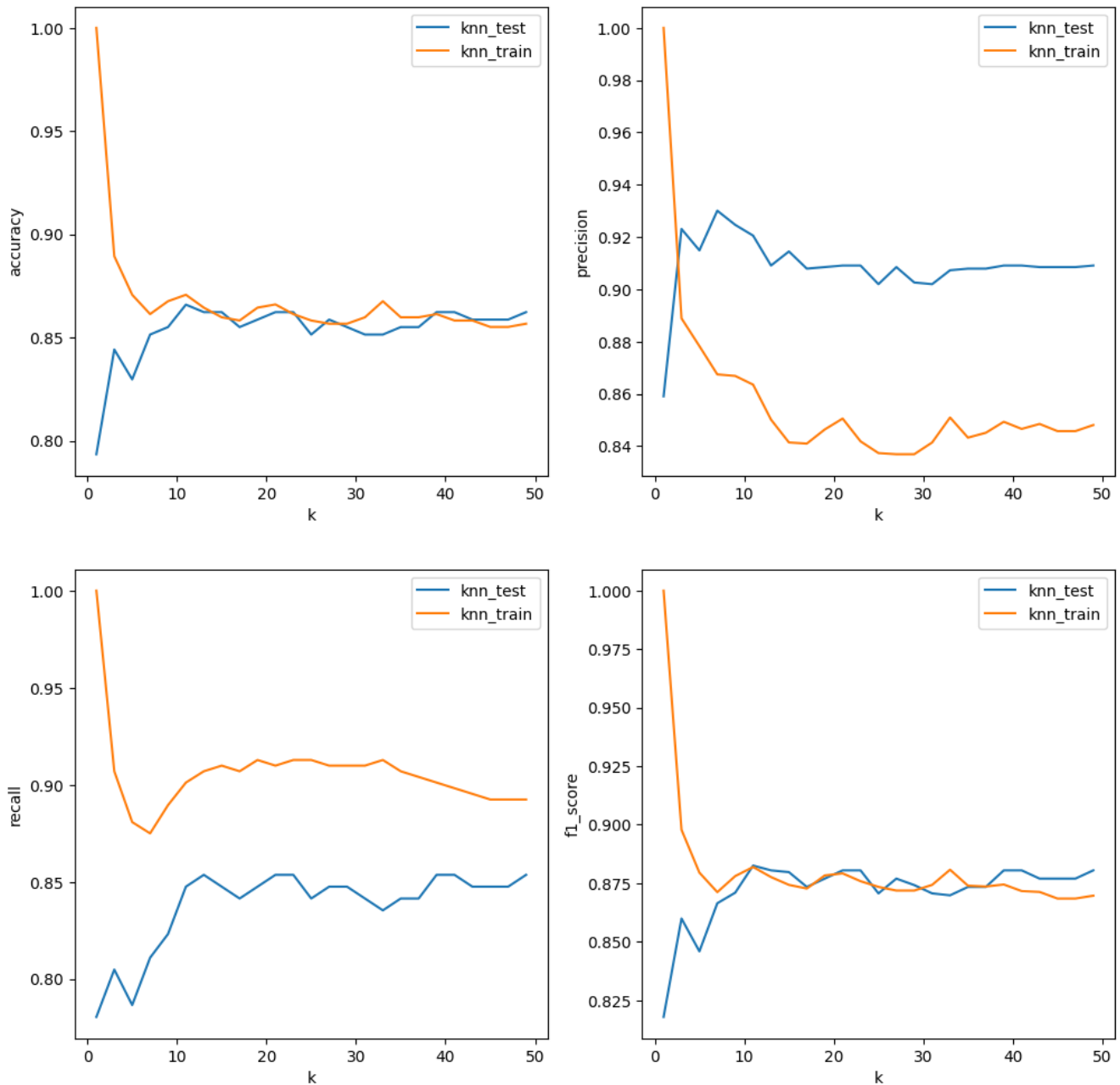
L'algorithme des k plus proches voisins (K-NN) est une méthode d'apprentissage supervisé utilisée pour la classification et la régression. Il se base sur la similarité des caractéristiques entre les données. En classification, K-NN attribue une étiquette à une observation en fonction de la majorité des étiquettes de ses voisins les plus proches (déterminés par une valeur k préalablement définie). L'algorithme mesure la distance entre les points de données en utilisant des mesures telles que la distance euclidienne.

---

## 2) Optimisation des hyper-paramètres

Dans le but d'optimiser notre algorithme, nous avons testé plusieurs K voisin (impair) (51) afin de voir lequel donnait les meilleurs résultats pour le f1-score.

Ci-dessous, le graphique comparatif :



---

On constate que pour le f1-score, le meilleur K est environ 19, lorsque l'on se base sur l'ensemble de test.

**K choisi :**

- 11

---

## **b) Random Forest**

### **1) Définition**

Random Forest est un algorithme d'apprentissage ensembliste utilisé pour la classification et la régression. Il combine plusieurs arbres de décision pour produire des prédictions plus robustes et précises. Chaque arbre de décision est entraîné sur un échantillon aléatoire du jeu de données, et lors de la prédiction, la classe majoritaire prédite par les arbres individuels est sélectionnée comme prédiction finale.

---

## 2) Optimisation des hyper-paramètres

### Paramètres considérés :

- `n_estimators` = [200, 400, 600, 800, 1000] # Nombre d'arbres dans la forêt aléatoire
- `max_depth` = [10, 20, 30, 40, 50, None] # Nombre maximum de niveaux dans l'arbre
- `min_samples_split` = [2, 5, 10] # Nombre minimum d'échantillons requis pour diviser un nœud
- `min_samples_leaf` = [1, 2, 4] # Nombre minimum d'échantillons requis à chaque nœud feuille
- `bootstrap` = [True, False] # Méthode de sélection des échantillons pour former chaque arbre

Pour trouver la meilleure combinaison, nous avons utilisé GridSearchCV.

GridSearchCV est une technique utilisée en apprentissage automatique pour rechercher de manière systématique les meilleurs hyper-paramètres pour un modèle. Il s'agit d'une méthode de recherche d'hyper-paramètres qui permet d'évaluer plusieurs combinaisons de valeurs d'hyper-paramètres pour un modèle donné.

### Meilleurs Paramètres :

- **`bootstrap` = True**
- **`max_depth` = 40**
- **`min_samples_leaf` = 1**
- **`min_samples_split` = 10**
- **`n_estimators` = 200**

---

## c) Régression Logistique

### 1) Définition

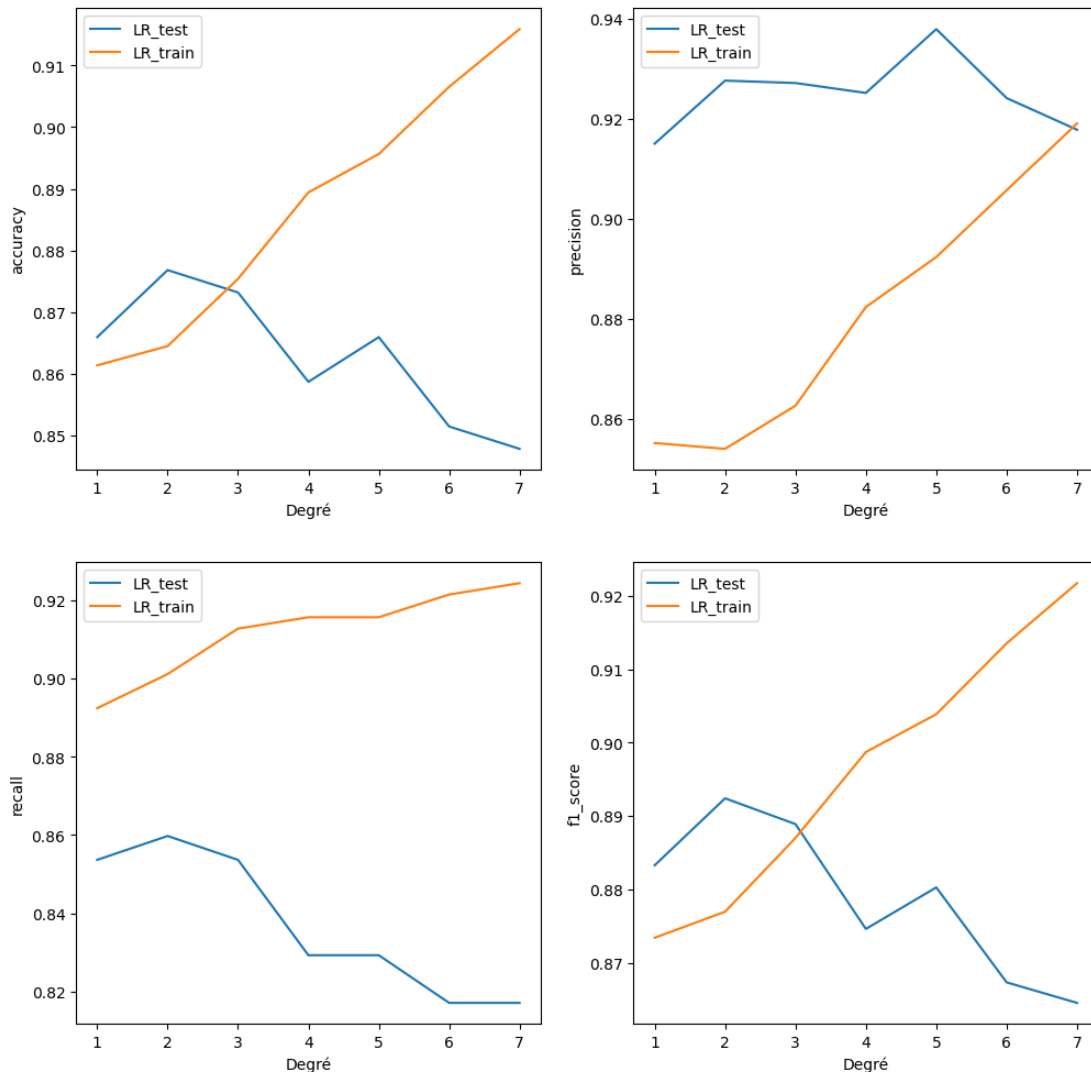
La Régression Logistique est un algorithme d'apprentissage supervisé utilisé pour la classification binaire. Contrairement à son nom, elle est utilisée pour résoudre des problèmes de classification plutôt que de régression. La Régression Logistique modélise la probabilité qu'une observation appartienne à une classe particulière. Elle utilise une fonction logistique pour estimer la probabilité et une fois que la probabilité est estimée, un seuil est utilisé pour classer les observations.

## 2) Optimisation des hyper-paramètres

Dans un premier temps, on va déterminer le meilleur degré à prendre pour notre régression logistique.

On a fait varier le degré de 1 à 8. Nous prenons pour ce test un  $C = 1$  et un nombre d'itérations de 2000.

Ci-dessous, le graphique des résultats :



**D'après le graphique, le meilleur degré à choisir est 2.**

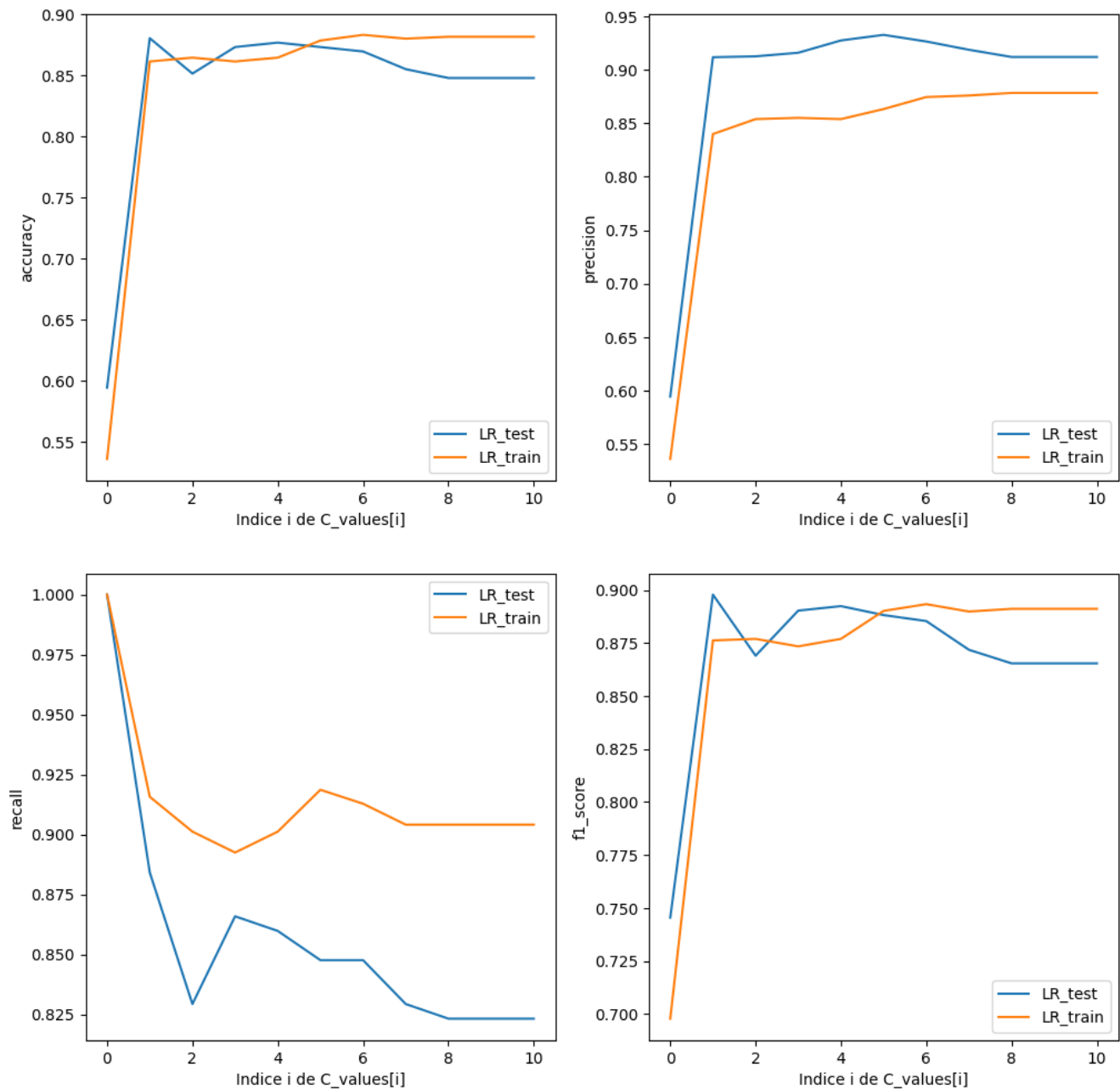


---

Ensuite, nous allons faire varier le  $C$  :

- $C\_values = [0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000, 10e4, 10e6, 10e9]$

Ci-dessous, le graphique des résultats :



D'après le graphique le meilleur  $C$  se trouve à l'indice 1, soit 0.001.

---

## **d) Réseau de neurones**

### **1) Définition**

Les réseaux de neurones sont des modèles d'apprentissage profond qui imitent le fonctionnement du cerveau humain pour apprendre à partir des données. Ils sont constitués de couches de neurones interconnectés, chaque neurone effectuant des opérations mathématiques sur les entrées et les transmettant aux couches suivantes. Les réseaux de neurones peuvent résoudre des problèmes complexes et non linéaires. Ils sont utilisés dans divers domaines tels que la vision par ordinateur, le traitement du langage naturel, la prédiction, et nécessitent souvent des ensembles de données volumineux pour être efficaces.

---

## 2) Optimisation des hyper-paramètres

Nous avons commencé par instancier un optimizer 'Adam'. Adam (Adaptive Moment Estimation) est un algorithme d'optimisation populaire utilisé dans l'entraînement de réseaux de neurones. Il est une extension de l'algorithme SGD (Stochastic Gradient Descent). Son principal atout : taux d'apprentissage adaptatif et séparé pour chaque poids.

Ensuite, nous avons instancié un 'EarlyStopping' pour éviter le surapprentissage.

Avec cela, nous avons instancié un pré-modèle pour effectuer nos tests avec ces paramètres :

- KerasClassifier
  - `model = create_Node_model` # Voir le code pour avoir le détail de la fonction
  - `activation = "relu"` # Pour l'instanciation, va varier ensuite
  - `neurons_per_layer = (10, )` # Pour l'instanciation, va varier ensuite
  - `validation_split = 0.3` # Division en deux ensembles : Entraînement et Validation (70/30%)
  - `epochs = 10` # Nombre d'itérations complètes sur toutes les données d'entraînement pour entraîner le modèle
  - `optimizer = optimizer` # Algorithme d'optimisation
  - `loss = 'binary_crossentropy'` # Fonction de perte
  - `verbose = 2` # Messages console
  - `random_state = 42` # Reproductibilité
  - `callbacks = [early_stopping_monitor]` # Fonctions à exécuter à un moment

Nous avons défini une liste d'hyper-paramètres à tester avec la validation croisée 'GridSearchCV' :

- 
- activation # fonction sigmoïd pour la dernière couche car classification binaire
    - relu
    - elu
    - tanh
  - neurons per layer # sans la dernière couche qui possède 1 neurone
    - (10, )
    - (10, 5, )
    - (20, )
    - (20, 5, )
    - (50, 10, )
    - (50, 20, 5, )
    - (100, 10, )
    - (100, 25, 5, )
    - (200, 50, 10, )
    - (500, 100, 25, 5, )
    - (918, )
    - (918, 918, )
    - (918, 918, 918, )
    - (918, 300, 300, )
    - (918, 300, 300, 300, )
    - (918, 230, 230, )
    - (918, 450, )
    - (918, )
    - (918, 300, )
    - (918, 450, 450, )
    - (918, 230, 230, 230, 230, )
    - (918, 450, 220, 100, 50, )

**À l'issue de la validation croisée 'GridSearchCV', nous avons obtenu :**

- 
- **Meilleurs Paramètres :**
    - 'activation': 'tanh', 'sigmoid'
    - 'neurons\_per\_layer': (918, 1)

---

## e) SVC (Support Vector Machine)

### 1) Définition

Les Machines à Vecteurs de Support (SVM) ou Support Vector Machines sont des modèles d'apprentissage supervisé utilisés pour la classification et la régression. En classification, les SVM cherchent à trouver un hyperplan qui sépare au mieux les différentes classes d'observations. L'objectif est de maximiser la marge entre les classes, en utilisant les observations les plus proches de la frontière de décision (vecteurs de support). Les SVM peuvent également utiliser des noyaux pour gérer des données non linéairement séparables en les projetant dans un espace de dimension supérieure. Ils sont efficaces pour les jeux de données de taille moyenne à petite et peuvent être sensibles au choix du noyau et aux paramètres de régularisation.

---

## 2) Optimisation des hyper-paramètres

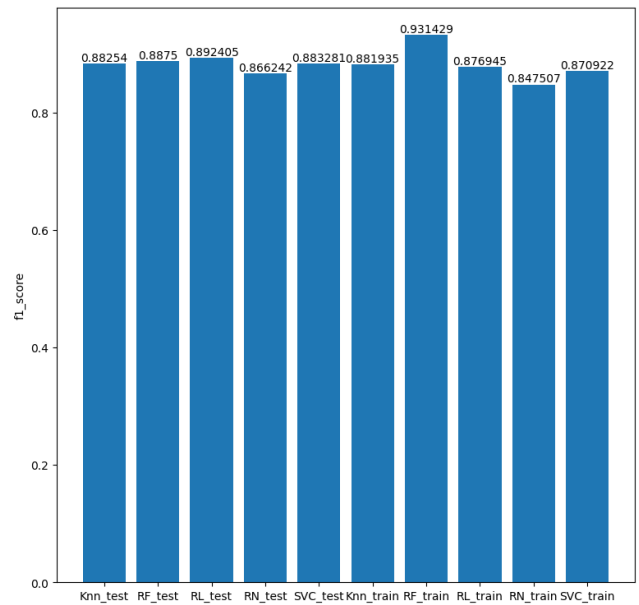
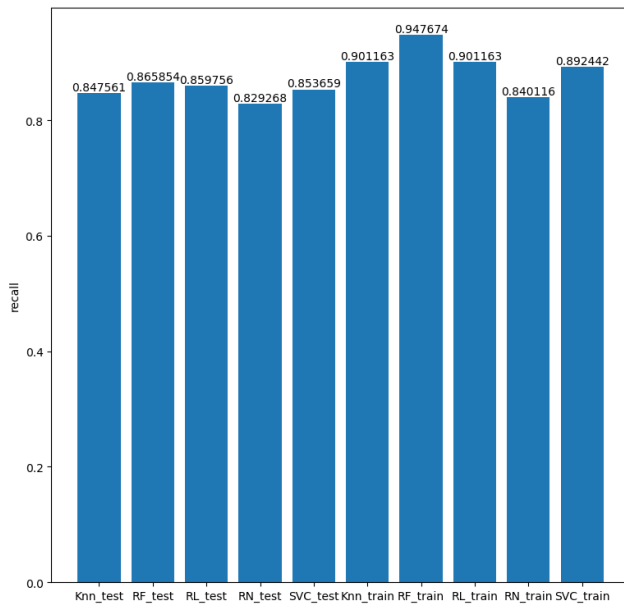
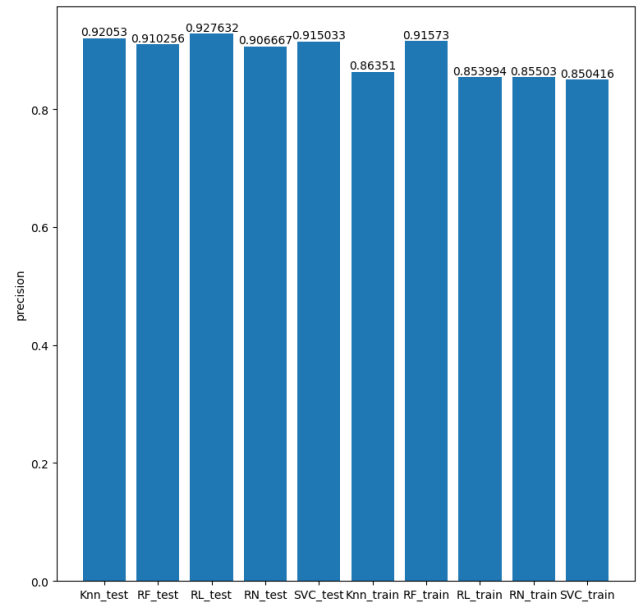
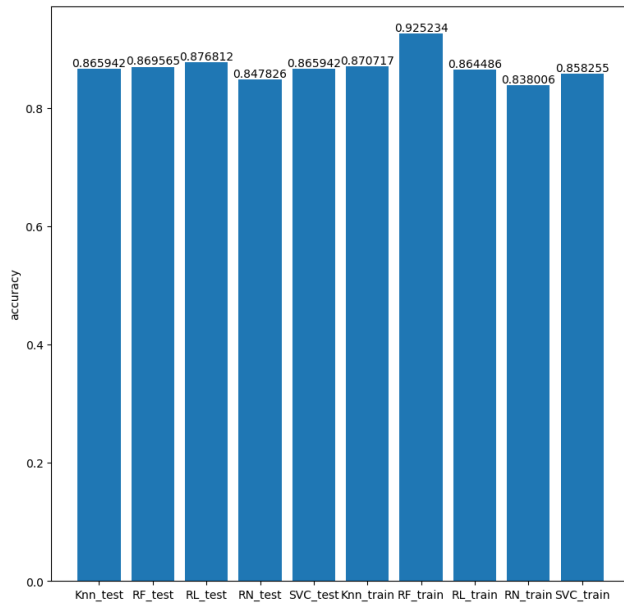
Pour l'optimisation de cet algorithme, nous avons dressé ce tableau de paramètres :

- param\_grid
  - 'C': [0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000, 10e4, 10e6, 10e9]
    - # Paramètre de régularisation (C) : plus C est grand, plus la régularisation est faible
  - 'kernel': ['linear', 'rbf', 'poly']
    - # Type de noyau (kernel) : linéaire, gaussien ou polynomial
  - 'gamma': ['scale', 'auto']
    - # Coefficient du noyau pour 'rbf' et 'poly':
      - 'scale' =  $1 / (n\_features * X.var())$
      - 'auto' =  $1 / n\_features$

Encore une fois grâce à la validation croisée 'GridSearchCV', voici les meilleurs paramètres trouvés :

- **Meilleurs Paramètres**
  - 'C': 100000.0
  - 'gamma': 'scale'
  - 'kernel': 'linear'

## V - Résultats obtenus





---

## VI - Analyse des résultats

Dans cette analyse, nous ne tiendrons compte que de la métrique sélectionnée (cf. partie IV) afin d'assurer la cohérence de nos propos. Mais vous pouvez regarder de plus près le graphique récapitulatif (cf. partie V) si vous souhaitez voir les résultats pour d'autres métriques.

Tableau récapitulatif des résultats :

| Ensemble du dataset | Classement | Algorithme            | f1-score |
|---------------------|------------|-----------------------|----------|
| <u>Entraînement</u> | 1          | Random Forest         | 0.931429 |
|                     | 2          | K-NN                  | 0.881935 |
|                     | 3          | Régression Logistique | 0.876945 |
|                     | 4          | SVC                   | 0.870922 |
|                     | 5          | Réseau de neurones    | 0.847507 |
|                     |            |                       |          |
| Ensemble du dataset | Classement | Algorithme            | f1-score |
| <u>Test</u>         | 1          | Régression Logistique | 0.892405 |
|                     | 2          | Random Forest         | 0.8875   |
|                     | 3          | SVC                   | 0.883281 |
|                     | 4          | K-NN                  | 0.88254  |
|                     | 5          | Réseau de neurones    | 0.866242 |

Nous remarquons que l'algorithme le plus efficace concernant les données de test est celui de la régression logistique, avec près de 90% de f1-score.

---

Le résultat du réseau de neurones est étonnant, étant dernier parmi tous les algorithmes, que ce soit sur le f1-score ou les autres. Cela peut provenir du fait qu'on ne fait varier que deux paramètres différents dans le GridSearchCV. On a d'ailleurs remarqué que ses résultats dépendent beaucoup de la seed, pouvant le faire passer du pire au meilleur algorithme, ce qui n'est pas normal.

Les quatre autres algorithmes sont très serrés en termes de résultats, et stables.

Tous les algorithmes testés et développés ont un résultat satisfaisant.

En affinant encore plus nos hyper-paramètres et au fruit de longues heures de patience supplémentaires, nous pensons que les résultats pourraient être encore meilleurs.

---